CAHIER DES CHARGES

Application intégrée de gestion et de pilotage d’une imprimerie

Backend : Laravel **Applications :** Flutter Desktop et Flutter Mobile

Version : cadrage fonctionnel préliminaire

Objectif : concevoir un système centralisé, robuste et évolutif couvrant les activités de l’imprimerie, du premier contact client jusqu’à la production, au suivi financier et à l’administration du personnel.

Document de travail à valider avec la direction et les responsables de chaque service avant la conception détaillée.

Principe directeur : l’application doit s’adapter à l’organisation réelle de l’entreprise. Les catégories, champs, unités, règles métier et permissions qui doivent être configurables seront gérés par les personnes habilitées, plutôt que figés arbitrairement dans le code.

Sommaire

1. Présentation et contexte
2. Vision et objectifs
3. Principes de conception et périmètre
4. Utilisateurs, rôles et permissions
5. Parcours d’une commande
6. Modules fonctionnels
7. Traitement intelligent des fichiers
8. Architecture technique
9. Robustesse, sécurité et continuité
10. Migration depuis Aronium Lite
11. Interfaces et expérience utilisateur
12. Phasage de réalisation
13. Critères de validation
14. Points à étudier avec les responsables
15. Glossaire

Ce document distingue les besoins identifiés, les propositions et les éléments à confirmer. Il ne fige pas les processus internes qui doivent être décidés par les responsables.

1. Présentation et contexte

L'entreprise utilise actuellement Aronium Lite pour une partie de l'enregistrement des ventes et des tickets. Le travail opérationnel fait également intervenir WhatsApp, des dossiers partagés, les logiciels de production et des fiches ou documents séparés. Cette organisation peut entraîner des manipulations répétitives, des doubles saisies, des recherches de fichiers et des pertes d'information.

- Le client contacte notamment l'entreprise par WhatsApp, explique sa demande et transmet éventuellement un fichier.
- Le secrétariat clarifie la demande, confirme les modalités et enregistre actuellement la vente dans Aronium Lite.
- Des tickets sont imprimés pour le client et pour la production ; le fichier est transmis au service concerné.
- Les équipes récupèrent les fichiers, les préparent avec leurs logiciels et réalisent les travaux.
- Les données de production, consommations, pertes et encaissements alimentent ensuite le suivi et les rapports.
- Les besoins couvrent aussi le marketing, la comptabilité, le stock, les ressources humaines, les équipements et l'administrationgénérale.

Le futur système ne doit pas être une copie d'Aronium Lite. Il doit reprendre les fonctions utiles après analyse et mieux coordonner les services, suivre les opérations de bout en bout et réduire le temps de traitement des fichiers.

2. Vision et objectifs

Mettre en place un système central de gestion, accessible sur ordinateur et mobile, reposant sur Laravel pour le backend et Flutter pour les interfaces.

- Centraliser les clients, demandes, commandes, travaux, fichiers, paiements, stocks, employés et rapports.
- Réduire les ressaisies, transferts manuels, recherches et validations répétitives.
- Accélérer la préparation des fichiers selon le document et le service demandé.
- Suivre le responsable, le statut, les délais, les blocages, la production et la livraison de chaque travail.
- Outiller l'administration : employés, présence, absences, rémunération, demandes et indicateurs, selon les règles validées.
- Permettre aux responsables habilités de définir les articles, unités, catégories, seuils et règles de stock.
- Protéger les données par des permissions, une traçabilité, des sauvegardes et une procédure de reprise.
- Ajouter de nouveaux services et paramètres sans reconstruire l'application.

3. Principes de conception et périmètre

- **Paramétrage par les responsables :** catégories, unités, services, champs et règles simples doivent être modifiables par les personnes habilitées.
- **Permissions granulaires :** séparer consulter, créer, modifier, valider, exporter, clôturer, supprimer et administrer.
- **Responsabilités distinctes :** les actions sensibles peuvent nécessiter un droit spécifique ou une approbation.
- **Traçabilité :** les changements importants doivent conserver l'auteur, la date, l'objet et, si nécessaire, le motif.
- **Pas de règles arbitraires :** les méthodes comptables, de paie, de valorisation du stock et de production seront définies avec les responsables.
- **Évolution progressive :** livrer d'abord un noyau fiable autour des commandes, fichiers et travaux, puis étendre les modules.

3.1 Domaines envisagés

Domaine	Couverture indicative
Commercial	Demandes, clients, devis éventuels, commandes et historique.
Production	Ordres de travail, affectation, statut, contrôle et livraison.
Fichiers	Réception, stockage, versions, analyse, préparation et archivage.
Finance	Encaissements, paiements, soldes, dépenses et rapports à définir.
Stock	Articles configurables, mouvements, consommations, pertes et inventaires.
Ressources humaines	Dossiers employés, pointage, absences, demandes et rémunération.
Administration	Utilisateurs, rôles, permissions, paramètres, journaux et sauvegardes.
Pilotage	Tableaux de bord et rapports adaptés aux responsabilités.

4. Utilisateurs, rôles et permissions

Les profils ci-dessous reflètent les acteurs identifiés à ce stade. La liste est à confirmer : tous les employés n'ont pas forcément besoin des mêmes écrans, ni même d'un accès direct à toutes les fonctions.

- **Directeur général :** vision globale, indicateurs et validations selon la politique interne.
- **Responsable marketing et administratif :** suivi commercial, administratif et coordination.
- **Marketing :** demandes entrantes, prospects, relances et suivi des réponses.
- **Comptabilité :** opérations financières et rapports autorisés.
- **Secrétariat :** accueil des demandes, dossiers clients, commandes et transmission des travaux.
- **Graphiste :** tâches graphiques, fichiers source, versions et préparation.
- **Machiniste et aide-machiniste :** file de production, consignes, quantités, consommations et résultats selon leurs droits.
- **Chargé du développement numérique et de l'appui technique :** maintenance et assistance technique selon un accès délégué.
- **Responsable de stock ou autres profils :** à créer si l'organisation le confirme.

4.1 Modèle de permissions

- Utiliser des rôles regroupant des permissions, attribués aux comptes nominatifs.
- Prévoir des restrictions supplémentaires par service, équipe, période ou type de document si nécessaire.
- Séparer les droits de consulter les salaires, modifier un salaire et valider une paie.
- Séparer les droits d'effectuer un mouvement de stock, de corriger un inventaire et d'approuver une correction.
- Permettre la désactivation d'un compte sans effacer l'historique des opérations qu'il a réalisées.
- Un utilisateur ne doit jamais pouvoir s'accorder lui-même des droits supérieurs.

Les permissions doivent être vérifiées par Laravel côté serveur. Masquer un bouton dans Flutter ne constitue pas une protection suffisante.

5. Parcours global d'une commande

- **Réception :** demande par WhatsApp, email, téléphone, accueil ou autre canal configuré.
- **Qualification :** identifier client, service, dimensions, quantité, délai, fichiers et informations manquantes.
- **Évaluation commerciale :** prix ou devis selon les règles autorisées.
- **Confirmation :** création de commande et enregistrement de l'acompte ou du paiement si applicable.
- **Création du travail :** une commande peut générer plusieurs travaux, affectés à un service ou une équipe.

- **Traitement du fichier :** analyse, contrôle, préparation et mise à disposition du service.
- **Production :** opérateurs consultent la file de travaux et mettent à jour les étapes autorisées.
- **Contrôle et livraison :** enregistrer les quantités, les reprises, le contrôle final et la remise au client.
- **Clôture :** compléter les informations utiles au suivi financier, aux consommations et aux rapports.
- **Historique :** conserver les versions, responsables, dates, statuts et résultats.

5.1 Statuts et exceptions

Les statuts exacts seront définis avec les services. Proposition initiale : reçu, à qualifier, en attente d'informations, devis en cours, en attente de confirmation, confirmé, en préparation, en production, en contrôle, prêt, livré, clôturé, suspendu ou annulé. Prévoir aussi les cas de fichier manquant, dimensions incohérentes, paiement incomplet, reprise, urgence, panne et perte de connexion.

6. Modules fonctionnels

6.1 Clients, demandes et suivi commercial

- Fiche client avec coordonnées et historique des demandes, commandes, paiements et fichiers selon les droits.
- Enregistrement du canal d'origine : WhatsApp, email, téléphone, accueil ou autre.
- Suivi des demandes non converties en commande, relances, responsable et prochaine action.
- Historique des échanges et pièces jointes sans imposer à tous les clients d'utiliser une application.
- Intégration WhatsApp ou email à étudier selon faisabilité, coût et autorisations.

6.2 Catalogue des services et tarifs

- Catalogue administrable des prestations réelles de l'entreprise, sans imposer une liste figée.
- Champs par prestation : unité, dimensions, quantité, support, finition, couleur, délai ou autres caractéristiques.
- Règles de prix configurables : prix fixe, unité, surface, quantité, options ou calcul spécifique à valider.
- Historique des changements de prix et droits spécifiques pour modifier les tarifs.
- Ne pas considérer les unités, noms et prix d'Aronium comme définitifs sans validation.

6.3 Commandes et ordres de travail

- Référence unique pour chaque commande et travail.
- Une commande peut contenir plusieurs tâches ou étapes si plusieurs services interviennent.
- Affectation d'un service, opérateur ou équipe, priorité, échéance et consignes.
- Dimensions confirmées, quantité, finition et fichier validé visibles dans les instructions.
- Historique des changements et notifications aux personnes concernées.
- Génération éventuelle d'un ticket client et d'un bon interne, si cette pratique est confirmée.

6.4 Fichiers et documents

- Associer chaque fichier à une demande, un client ou une commande.
- Conserver l'original reçu ; chaque version préparée doit être distincte et identifiable.
- Enregistrer format, taille, pages, métadonnées utiles, empreinte du fichier et état du traitement.
- Proposer un classement automatique par commande et service, avec correction humaine possible.
- Historique des versions, validations et remplacements.
- Politique de conservation, archivage et suppression à définir selon le volume et les besoins.

6.5 Production

- File de travail par service : à faire, en cours, bloqué, à contrôler et terminé.
- Instructions de production adaptées à chaque type de travail.
- Saisie de quantités produites, temps passé si pertinent, consommations et pertes selon les règles du service.

- Déclaration d'incident, défaut, reprise ou panne avec commentaire.
- Contrôle final et identification de la personne qui valide.
- Les fiches de production doivent être configurables par service plutôt qu'identiques pour toutes les machines.

6.6 Finance et suivi des paiements

- Enregistrer montants, acomptes, paiements partiels, soldes, remboursements et annulations selon les règles validées.
- Moyens de paiement configurables et historique des opérations.
- Rapports par période, service, responsable ou type d'opération selon les droits.
- Distinguer chiffre d'affaires, encaissements, créances, dépenses et coûts estimés.
- Confirmer avec la comptabilité les exports et obligations réglementaires ; ne pas présumer que l'application remplace un logicielcomptable.

6.7 Stocks et consommables

- Les responsables habilités définissent les catégories, articles, unités et caractéristiques suivis.
- Exemples seulement : encres, rouleaux de bâche, vinyle, feuilles DTF, supports, pièces et consommables divers.
- Opérations possibles : entrées, sorties, transferts, réservations, retours, pertes, corrections et inventaires.
- Seuils d'alerte, emplacements et fournisseurs configurables si souhaités.
- Chaque mouvement conserve date, auteur, quantité, motif, document lié et approbation éventuelle.
- Comparer consommation théorique et réelle lorsque cela est utile et mesurable.
- Les règles d'unité, coût et valorisation sont définies par les responsables, pas imposées dans le code.

6.8 Ressources humaines et administration du personnel

- Dossiers employés : service, poste, date d'entrée, statut et documents autorisés.
- Pointage d'arrivée et de départ, horaires, retards, absences et anomalies selon la méthode retenue.
- Demandes d'absence ou de congé, période, motif, justificatif éventuel, validation et solde si applicable.
- Éléments de rémunération : salaire de base, primes, commissions, retenues et avances selon les règles définies.
- Préparation, validation et historique de la paie, avec exports nécessaires.
- Indicateurs de productivité définis avec les responsables ; ne pas assimiler automatiquement le nombre de tâches à laperformance réelle.
- Un responsable des présences n'a pas automatiquement accès aux salaires.
- Les calculs légaux et documents obligatoires devront être validés par la direction ou un professionnel compétent.

6.9 Équipements et maintenance

- Registre configurable des machines, ordinateurs, imprimantes, réseau et autres équipements suivis.
- Historique des pannes, interventions, pièces changées, coûts et prestataires si nécessaire.
- Planification de maintenance préventive et signalement d'indisponibilité.
- Relier éventuellement les incidents aux travaux retardés.

6.10 Administration de l'application

- Gestion des utilisateurs, rôles, permissions, services et paramètres.
- Configuration des champs et listes prévus comme modifiables sans code.
- Journaux d'audit, sessions, réinitialisation d'accès et désactivation des comptes.
- Supervision des transferts, tâches de fond, erreurs et espace de stockage.
- Sauvegardes et exports selon les droits.

6.11 Tableaux de bord et rapports

- Tableau de bord adapté à chaque rôle.

- Indicateurs possibles : commandes en attente, délais, travaux bloqués, livraisons, encaissements, stocks bas, absences etincidents.
- Filtres par période, service, responsable, statut et type de travail.
- Exports PDF, CSV ou tableur selon le besoin.
- Les définitions de chaque indicateur sont à valider pour éviter les chiffres ambigus.

7. Traitement intelligent des fichiers

La réduction du temps consacré aux fichiers est une priorité. Le système doit limiter les recherches et manipulations inutiles, sans modifier silencieusement l'original ni lancer une production à partir d'une détection incertaine.

7.1 Parcours proposé

- Réception du fichier et création rapide d'une référence de traitement.
- Analyse en arrière-plan : format, taille, nombre de pages, dimensions de page et métadonnées accessibles.
- Comparaison des caractéristiques détectées avec le service, les dimensions attendues et les consignes de commande.
- Détection et signalement des fichiers illisibles, formats non pris en charge ou dimensions inattendues.
- Création d'un aperçu pour identifier le document sans ouvrir systématiquement un logiciel lourd.
- Proposition de classement et de profil de préparation ; validation humaine lorsque nécessaire.
- Création d'une copie de travail, puis mise à disposition du service concerné.
- Traçabilité des résultats, avertissements, versions et validations.

7.2 Automatisations à étudier

- Détection du format, des dimensions et des documents multipages.
- Création d'aperçus et miniatures.
- Signalement d'incohérences entre dimensions commandées et dimensions détectées.
- Profils de préparation configurables par type de service.
- Traitement asynchrone des gros fichiers pour que l'interface reste utilisable.
- Reprise des transferts interrompus et contrôle d'intégrité lorsque le stockage le permet.

Limite : la détection automatique ne remplace pas les règles de prépresse ni le jugement du graphiste ou du machiniste. Mise à l'échelle, imposition, découpe, conversion colorimétrique et fonds perdus doivent être spécifiés par type de travail puis testés avec les logiciels et machines réels.

7.3 Fichiers volumineux et réseau instable

- L'enregistrement d'une commande ne devrait pas attendre le transfert complet d'un gros fichier si l'architecture asynchrone lepermet.
- Afficher les états : en attente, transfert, reçu, analyse, prêt, erreur ou à vérifier.
- Ne pas stocker les gros fichiers directement dans les tables SQL : conserver les métadonnées et une référence au stockage.
- Définir les limites de taille, quotas, durée de conservation et archivage après mesure des volumes.
- Protéger les fichiers par authentification et autorisation ; éviter les liens publics permanents.

8. Architecture technique cible

L'architecture sépare les interfaces, la logique métier, les données structurées, les fichiers et les traitements longs. Les versions et l'hébergement exacts seront choisis après étude de l'infrastructure, du budget et des contraintes.

Couche	Responsabilité
Flutter Desktop	Interface des postes fixes : secrétariat, production, comptabilité et administration selon le profil.
Flutter Mobile	Consultation, notifications, demandes et validations adaptées au mobile.
API Laravel	Authentification, autorisation, validation des données et échanges sécurisés.
Services métier Laravel	Commandes, workflows, stock, RH, finances, audit, notifications et intégrations.
Base SQL	Clients, utilisateurs, commandes, statuts, mouvements, métadonnées, paramètres et historiques.
Stockage de fichiers	Originaux, versions de travail, aperçus et archives, séparés de la base SQL.
File de tâches / workers	Analyse de fichiers, aperçus, notifications, imports et exports en arrière-plan.
Couche	**Responsabilité**
Supervision	Journaux, alertes, capacité, sauvegardes et tests de restauration.

8.1 Backend Laravel

- Organiser le code par domaines métier et responsabilités ; éviter une logique métier concentrée dans les contrôleurs.
- Séparer API, contrôleurs, services métier, politiques d'autorisation, validations, modèles, événements, tâches asynchrones ettests.
- Versionner l'API pour faire évoluer Flutter sans rupture non maîtrisée.
- Valider toutes les permissions côté serveur, pas seulement dans l'interface.
- Utiliser des transactions SQL pour les opérations qui doivent être atomiques, notamment certains paiements et mouvements destock.
- Prévoir documentation API, migrations, tests automatisés et environnements distincts de développement, test et production.

8.2 Base de données

- Choisir le moteur SQL après vérification de l'hébergement, des compétences et des besoins ; PostgreSQL est une optionsérieuse, à confirmer.
- Structurer les relations, contraintes et index pour limiter incohérences et lenteurs.
- Ne pas coder en dur les catégories de stock, unités, services et champs qui doivent rester configurables.
- Concevoir les champs personnalisables avec prudence pour garder des rapports fiables et des données contrôlables.

8.3 Flutter Desktop et Mobile

- Les deux applications utilisent la même API Laravel et les mêmes règles métier côté serveur.
- Adapter les écrans au contexte : saisie et listes sur ordinateur, consultation et validations courtes sur mobile.
- Prévoir les états réseau, erreurs, sessions expirées et reprises d'action.
- Définir explicitement tout besoin hors ligne et la gestion des conflits avant de promettre une synchronisation.

8.4 Hébergement et stockage

- Comparer l'hébergement actuel, un VPS/serveur dédié et un stockage objet selon coûts, volume, vitesse, disponibilité etlocalisation.
- Ne pas présumer qu'un compte cloud spécifique existe : vérifier l'infrastructure actuelle avant de décider.
- Pour un stockage local ou hybride, définir les accès réseau, les sauvegardes hors site et le comportement en cas de coupureInternet.

- Mesurer volume quotidien, taille maximale et durée de conservation avant de dimensionner.

9. Robustesse, sécurité et continuité

La robustesse repose sur la cohérence des données, la sécurité des accès, les tests, les sauvegardes et des procédures de reprise documentées.

9.1 Sécurité

- Authentification sécurisée, sessions maîtrisées et réinitialisation de mot de passe.
- Vérification des permissions pour chaque action et ressource sensible côté API.
- Validation des fichiers téléversés, formats autorisés, limites de taille et protections adaptées.
- Chiffrement des communications et protection des secrets techniques.
- Journalisation des connexions et opérations sensibles, avec accès restreint.
- Principe du moindre privilège pour la base, le stockage et les comptes techniques.

9.2 Intégrité et audit

- Références uniques pour commandes, travaux, transactions et mouvements.
- Annulation ou correction tracée plutôt que suppression silencieuse d'opérations sensibles.
- Historique des changements de prix, paramètres, permissions et données confidentielles.
- Protection contre les doublons lors d'une répétition d'action ou d'une erreur réseau.
- Validation des transitions importantes et contrôle des opérations concurrentes.

9.3 Sauvegardes et reprise

- Sauvegardes planifiées de la base et des fichiers avec rétention définie.
- Copie de sauvegarde séparée du serveur principal et contrôle des accès.
- Tests réguliers de restauration : une sauvegarde non testée n'offre pas une garantie suffisante.
- Définition du délai de reprise acceptable et de la perte de données maximale tolérable.
- Surveillance de l'espace disque, des erreurs, des tâches en attente et de l'état des sauvegardes.

9.4 Performance

- Exécuter les traitements longs en arrière-plan.
- Utiliser pagination, filtres et index pour les listes volumineuses.
- Prévoir reprises de tâches échouées et visibilité des erreurs pour les administrateurs.
- Tester les opérations fréquentes et les transferts de gros fichiers avant la mise en service.

10. Données et migration depuis Aronium Lite

La sauvegarde Aronium Lite examinée contient une base SQLite exploitable : 125 produits, 818 clients, 4 648 documents de vente et 5 277 lignes de documents, couvrant la période du 11 juin 2024 au 1er août 2026. Ces chiffres décrivent la sauvegarde analysée et ne constituent pas un état comptable certifié.

10.1 Constats à traiter

- Le catalogue comprend plusieurs familles de prestations d'impression et d'objets personnalisés.
- Les unités ne sont pas renseignées uniformément ; de nombreux produits n'ont pas d'unité renseignée.
- Certains noms sont similaires : aucune fusion automatique ne doit être faite sans validation.
- Le champ de coût n'est renseigné que sur une partie du catalogue ; sa signification doit être confirmée.
- Des quantités de stock négatives apparaissent ; elles doivent être expliquées avant de reprendre un stock comme quantitéphysique fiable.
- Dans la sauvegarde analysée, les documents observés sont des ventes, même si l'outil prévoit d'autres types de documents.

10.2 Stratégie de migration

- Conserver une copie intacte et privée de la source.
- Vérifier avec les utilisateurs la signification des champs et les pratiques réelles d'Aronium.
- Définir le modèle cible et les règles de nettoyage du catalogue.
- Effectuer un import test avec rapport des données importées, transformées et à vérifier.
- Comparer un échantillon de clients, produits, documents, dates et montants.
- Faire valider les résultats par les responsables avant la bascule et conserver une stratégie d'archivage.

La migration ne doit pas reproduire automatiquement les limites de l'ancien outil. Seules les données historiques utiles seront reprises selon un plan validé.

11. Interfaces et expérience utilisateur

- Interface sobre, rapide et lisible, adaptée à un usage quotidien en environnement de production.
- Menus et écrans adaptés au rôle, plutôt qu'une interface identique pour tous.
- Recherche rapide par référence, client, statut, date ou nom de fichier.
- Affichage clair des tâches en attente, erreurs, validations et blocages.
- Formulaires courts et champs conditionnels lorsque cela réduit réellement la saisie.
- Sur desktop : listes de travail, filtres, tableaux, aperçu et saisie efficace.
- Sur mobile : consultation, notifications, demandes et validations adaptées à l'écran.
- Messages d'erreur compréhensibles, sans exposer de données confidentielles.

11.1 Notifications

Prévoir des notifications configurables pour les nouvelles demandes, fichiers prêts, informations manquantes, tâches bloquées, validations attendues, stocks bas ou demandes d'absence. Les canaux seront choisis selon le coût, la connectivité et les préférences de l'entreprise.

12. Phasage de réalisation

Le développement doit être progressif : mieux vaut un noyau fiable, utilisé et testé, qu'un grand nombre de modules incomplets.

Phase	Travaux	Résultat attendu
0 — Cadrage	Entretiens, flux, règles, documents et contraintes techniques.	Spécifications détaillées validées.
1 — Socle	API Laravel, authentification, permissions, base SQL, audit, tests et sauvegardes.	Backend sécurisé et maintenable.
2 — Commandes et fichiers	Clients, demandes, commandes, fichiers, ordres de travail et statuts.	Flux central opérationnel.
3 — Production	Files de travail, analyse des fichiers, aperçus, versions et tâches asynchrones.	Moins de manipulations et meilleur suivi.
4 — Finance et stock	Paiements, mouvements, consommations, pertes et rapports après validation.	Suivi cohérent.
5 — RH et administration	Employés, pointage, absences, rémunération et validations.	Administration structurée.
6 — Pilotage	Tableaux de bord, exports, maintenance et intégrations complémentaires.	Solution étendue.

L'ordre pourra être adapté après les entretiens. Les fonctions ayant un impact financier, notamment paie et valorisation du stock, ne doivent être automatisées qu'après validation des règles et cas de test.

13. Critères de validation

- Chaque utilisateur n'accède qu'aux données et actions autorisées, y compris via l'API.
- Une commande est suivie de sa réception à sa clôture avec les responsables, dates et changements de statut.
- L'original d'un fichier est conservé ; les versions préparées sont distinctes et identifiables.
- Un transfert volumineux ou interrompu ne bloque pas toute l'application et peut être repris si cette fonction est prévue.
- Les champs et listes configurables peuvent être modifiés par un profil habilité sans changement de code.
- Les mouvements de stock et transactions sensibles sont traçables et protégés contre les doublons.
- Les données de salaire et actions de paie sont protégées par des permissions spécifiques.
- Les rapports reposent sur des définitions validées et sont comparés aux données de référence.
- Une sauvegarde est restaurée lors d'un test documenté.
- Les opérations principales disposent de tests automatisés et d'une procédure de déploiement documentée.

14. Points à étudier avec les responsables

Les éléments suivants doivent être confirmés avant de figer les écrans et les règles. Ils ne sont pas imposés dans ce document : ils constituent le programme de cadrage.

14.1 Organisation

- Qui peut créer, confirmer, annuler et clôturer une commande ?
- Quelles opérations exigent une validation de la direction ou d'un responsable ?
- Quels services interviennent selon les prestations ?
- Quels documents sont obligatoires à chaque étape ?

14.2 Production et prépresse

- Quels logiciels, machines, formats et profils sont réellement utilisés ?
- Quelles opérations de préparation peuvent être automatisées sans risque ?
- Comment confirmer dimensions, quantité, finition, fonds perdus et profils colorimétriques ?
- Quelles consommations et pertes chaque service peut-il mesurer ?

14.3 Finance et stock

- Quelle procédure s'applique aux acomptes, paiements partiels, crédits, remboursements et annulations ?
- Quels exports attend la comptabilité ?
- Qui définit unités, catégories, seuils et valorisation du stock ?
- Comment sont suivies les consommations, chutes et pertes ?

14.4 Ressources humaines

- Quel dispositif de pointage est souhaité : saisie, badge, appareil ou autre ?
- Qui valide absences, congés, heures supplémentaires, avances et paie ?
- Quels indicateurs de productivité sont pertinents par métier ?
- Quelles règles et obligations légales doivent être prises en compte ?

14.5 Infrastructure

- Où seront hébergés API, base et fichiers ?
- Quel débit réseau est disponible sur les postes de production ?
- Quel fonctionnement est nécessaire lors d'une coupure Internet ?
- Quel volume de fichiers est reçu chaque jour et combien de temps doit-on les conserver ?
- Quels systèmes d'exploitation et appareils doivent être supportés ?

15. Glossaire

Terme	Définition dans le projet
Commande	Engagement commercial regroupant la demande du client et ses conditions.
Ordre de travail	Instruction opérationnelle destinée à un service ou une équipe.
Original	Fichier tel que reçu, conservé sans modification.
Version de travail	Copie créée pour une étape précise du traitement.
RBAC	Modèle d’autorisations où les permissions sont regroupées dans des rôles.
API	Interface sécurisée entre les applications Flutter et Laravel.
Tâche asynchrone	Traitement en arrière-plan qui n’oblige pas l’utilisateur à attendre.
Journal d’audit	Historique des actions importantes et de leurs auteurs.
Configuration métier	Paramètres modifiables par des personnes autorisées sans changer le code.

Exigences concernant les technologies, les bibliothèques et les coûts

1. Principe général

L'application doit privilégier les technologies gratuites, open source et auto-hébergeables, lorsque celles-ci répondent correctement aux besoins fonctionnels, aux exigences de sécurité et aux contraintes de performance.

Laravel constitue le backend principal de l'application. Flutter constitue la technologie des interfaces desktop et mobile.

Python peut être intégré en complément lorsque ses bibliothèques apportent un avantage technique réel, notamment pour l'automatisation, l'analyse, la conversion ou le traitement spécialisé de fichiers.

L'utilisation d'une technologie supplémentaire doit être justifiée. L'objectif n'est pas de multiplier les langages, frameworks, services et dépendances, mais de choisir une architecture cohérente, maintenable et économiquement viable.

2. Interdiction d'ajouter des services payants sans validation

L'agent de développement ne doit pas intégrer automatiquement des API commerciales, des services cloud payants, des abonnements, des solutions propriétaires payantes ou des services facturés à l'utilisation.

Cela concerne notamment :

- Les API de messagerie et de communication, y compris WhatsApp Business.
- Les services externes de stockage de fichiers.
- Les API d'intelligence artificielle.
- Les services OCR, de conversion ou d'analyse de documents facturés à l'utilisation.
- Les plateformes de notifications, d'envoi de SMS ou d'e-mails payants.
- Les solutions de sauvegarde, de surveillance ou d'hébergement supplémentaires.

Aucun de ces services ne doit être considéré comme obligatoire par défaut.

Si une fonctionnalité nécessite réellement un service externe payant, l'agent doit d'abord présenter le besoin, les avantages, les limites, les coûts éventuels et les solutions gratuites ou auto-hébergeables disponibles. L'intégration ne pourra avoir lieu qu'après validation explicite.

3. Bibliothèques et outils open source

Avant de développer une fonctionnalité complexe, l'agent doit rechercher les bibliothèques existantes adaptées au besoin.

Il doit privilégier les solutions :

- Gratuites et activement maintenues.
- Compatibles avec un usage professionnel et commercial.
- Dont la licence autorise l'utilisation envisagée.
- Disposant d'une documentation suffisante.
- Compatibles avec l'environnement de déploiement prévu.
- Ne nécessitant pas obligatoirement un abonnement ou une clé API payante.

Les dépendances doivent être limitées au nécessaire. Leur utilité, leur licence et leurs éventuelles contraintes doivent être documentées.

4. Traitement des fichiers d'imprimerie

L'application doit rechercher en priorité des solutions locales permettant de traiter les fichiers sans transmettre systématiquement les documents des clients à des plateformes tierces.

Les traitements envisagés comprennent notamment :

- La détection du format et du type de fichier.
- L'extraction des dimensions, du nombre de pages et des métadonnées disponibles.
- La génération d'aperçus et de miniatures.
- La vérification des caractéristiques nécessaires à la production.

- Le classement et l'association des fichiers aux commandes et aux services concernés.
- La conversion et les traitements automatisés compatibles avec les formats utilisés par l'imprimerie.

Laravel doit orchestrer ces traitements. Des bibliothèques PHP, des outils système ou des scripts Python peuvent être utilisés selon le besoin réel.

Les traitements lourds doivent être exécutés en arrière-plan lorsque cela est pertinent, avec suivi de progression, gestion des erreurs et possibilité de reprise.

5. WhatsApp et autres canaux de communication

L'application ne doit pas dépendre obligatoirement d'une API WhatsApp payante.

Le fonctionnement actuel de réception des fichiers par WhatsApp peut être conservé, avec une association manuelle ou semi-automatisée des fichiers aux commandes dans l'application.

Toute intégration officielle ou non officielle de messagerie doit être évaluée séparément, en tenant compte des coûts, des conditions d'utilisation, de la sécurité et de la fiabilité.

Une fonctionnalité ne doit pas être annoncée comme automatiquement intégrée si elle dépend d'un service externe qui n'a pas été configuré et validé.

6. Transparence sur les coûts

Avant toute décision technique ayant une incidence financière, l'agent doit distinguer :

- Les logiciels et bibliothèques gratuits.
- Les coûts d'hébergement et de stockage.
- Les coûts éventuels de maintenance et d'exploitation.
- Les services payants facultatifs.
- Les limites techniques des solutions gratuites.

Le caractère open source d'un logiciel ne signifie pas que son hébergement, son exploitation ou sa maintenance sont sans coût.

7. Obligation de justification technique

Pour chaque technologie ou service supplémentaire proposé, l'agent doit préciser :

1. Le problème concret qu'il résout.
2. Pourquoi Laravel seul ou les outils déjà retenus ne suffisent pas.
3. Les solutions gratuites disponibles.
4. Les éventuels coûts et contraintes de licence.
5. L'impact sur la maintenance et le déploiement.

La priorité reste une application robuste, sécurisée, évolutive et adaptée aux moyens disponibles, sans dépendances commerciales imposées inutilement.

Conclusion

Le projet vise à fournir à l'imprimerie un outil central de coordination et de gestion, de la demande client jusqu'à la production, aux consommations, aux encaissements et à l'administration du personnel. Laravel portera les règles métier, les permissions et la cohérence des données ; Flutter fournira des interfaces desktop et mobile adaptées aux usages de chaque profil.

La priorité est de réduire le temps perdu dans le traitement des fichiers et la circulation des informations, sans imposer des catégories, unités, règles de stock ou méthodes de gestion qui n'auraient pas été validées par les responsables. La prochaine étape sera de confronter ce cadrage aux pratiques de chaque service, puis de rédiger les spécifications détaillées et les scénarios de recette.

Fin du document — version préliminaire à compléter après validation des responsables.